

COMP2211 2023/24 LABORATORY MANUAL



School of Computing
University of Leeds
Leeds, LS2 9JT, United Kingdom

Version of September 25, 2023

Contents

1	Introduction	5
2	Week 1: Command line and xv6	7
2.1	Linux command line	7
2.1.1	Exercise 1	8
2.1.2	Exercise 2	8
2.1.3	Exercise 3	9
2.1.4	Exercise 4	9
2.1.5	Exercise 5	9
2.1.6	Exercise 6	9
2.1.7	Exercise 7	9
2.2	Reading and thinking activities	10
2.2.1	Exercise 8	10
2.2.2	Exercise 9	10
3	Week 2: Starting xv6	11
3.1	Getting a copy	11
3.2	Compiling the xv6 operating system	12
3.3	Xv6 shell	13
3.4	Exercise 1	13
3.4.1	Debugging a program	16
3.5	Exercise 2	16
3.6	Exercise 3	17
3.7	Exercise 4	18
3.8	Exercise 5	19
4	Week 3: Creating system calls and extending the xv6 shell	21
4.1	Creating new system calls	21
4.1.1	Step 1	22
4.1.2	Step 2	23
4.1.3	Step 3: Creating the user program	23
4.2	Exercise 1	23
4.3	Exercise 2	25

5	Week 4: Creating a small shell interface	27
6	Week 7: Exploring memory in regular Linux	31
6.1	Linux memory management	31
6.2	Exploring memory usage	32
7	Week 8: Exploring linked lists in C	35
7.1	Exercise	35
8	Weeks 9–11: Debugging dynamic memory management on xv6	37
8.1	Exercise 1: Debug malloc and free	37
8.2	Exercise 2: Implement optimisations	38
9	Conclusion	39

Chapter 1

Introduction

Welcome to COMP2211 23/24. This manual provides all of the practical activities for this module. Most of the exercises are *formative assessment*, which do not contribute to the final mark of the module. The weekly exercises are designed to help prepare for the final part, *summative assessment*, which constitutes 30% of the overall module mark. Staff in the laboratory can help you along the way and they have the model solutions which they can discuss with you after the dedicated assignment weeks.

We will be using Bash scripting, C programming, some git version control, and a small operating system designed for learning, called xv6. For Bash scripting and git we recommend using various materials online, as well as the `man` command in the terminal. For C programming we recommend *The C Programming Language (2nd Edition)* by Kernighan and Ritchie, 1988.

Acknowledgements

The exercises were created by utilizing previous years' COMP2211 material: we thank Martin Callaghan and Sam Wilson.

Chapter 2

Week 1: Command line and xv6

The purpose of this lab is to re-introduce you to the Linux command line and the Unix shell scripting language. There are many Unix shells that act as command-line interpreters, with many similar but also different features: `csh`, `sh`, `bash`, `zsh` and many others—look into the history. Open up the terminal on your Linux machine and run

```
$ echo "$SHELL"
```

or

```
$ echo "$0"
```

which should show which shell you are running.

The following set of exercises equip you (or provide a refresh) with foundational Unix shell skills that will be useful in this module.

Please type the commands into the terminal directly. Copying-pasting them from this PDF document might not always work.

2.1 Linux command line

First, get hold of the sample data we will be using today. Use a terminal on a Linux machine to run the following commands. Type them in rather than copy-paste if you encounter problems.

```
$ cd
$ git clone https://github.com/callaghanmt/shell-training.git
```

This will navigate to your Home directory folder and then download and create a new folder (called `shell-training`) full of other folders and data. Make sure you type in the `git clone` line exactly as shown. If not, you will get an error message.

If you would like to clone the `shell-training` files somewhere else rather than in the home directory, first create a new directory if needed and move there instead. For example

```
$ cd
$ mkdir work
$ cd $HOME/work
```

Here `$HOME` is an environment variable that stores an absolute path to your home directory.

Navigating directory tree: run `pwd` to determine where you are. Run `cd` to go to the home folder or `cd <path>` to go to a directory specified through a relative or absolute `<path>`. Run `cd -` (`cd` followed by a dash) to go back to where you were before you came to the current directory.

2.1.1 Exercise 1

Create the following directory tree in your home directory:

```
temp
temp/input_data/
temp/results/
temp/program/
```

- Create the file `input.txt` with a text editor, such as `nano` or `vim`, and populate it with some text. You may do it inside the terminal rather in the Graphical User Interface of your chosen text editor. In one command, move the file to `temp/input_data` and rename it to `control01.txt`.
- Create this directory tree in one line only: `temp/experiment/results/report`.
- Delete the work directory and all of its contents with one single command.

Write down the commands you have discovered, for your reference.

2.1.2 Exercise 2

Please meet three new commands:

- The `cat` command concatenates and prints files.
- The `sort` command will sort lines alphabetically or numerically.
- The `cut` command to split lines of text based on a given character e.g. `cut -d ',' -f 2` will split lines around the comma and give you the second part.

Use `man` command to get more information on other commands. For example, `man sort` will provide detailed information on `sort`. Exit with `q`.

Combine `cat`, `cut`, and `sort` to print out the Latin names from `birds.txt` in alphabetical order. Save the output to a new file.

2.1.3 Exercise 3

List *all the animals* on the Isle of Man alphabetically and find the 50th item in that list.

2.1.4 Exercise 4

`shell-training/data/` contains 300 data files, each of which should contain 100 values. One of these files is missing some data. Use a series of commands connected by **pipes** to identify the file with missing data (hint `wc -w` will tell you the number of values in a file, `sort -n` will sort numerically).

2.1.5 Exercise 5

Study the following Linux commands: `sed`, `grep` and `awk`.

- Use the `sed` command to delete the first character and last character in each line of a file (any file, you create it).
- Use the `grep` command to find how many lines of a file contain a given word. The filename and the word are provided as inputs.

2.1.6 Exercise 6

Write a shell script program and a C program to display “HELLO WORLD”. Compare the running time of the programs using `time` command. What do you observe?

2.1.7 Exercise 7

As a reminder, you can get various software packages on the School’s linux machines by using a command `module`. Start by having a look at what modules are available by running

```
$ module avail
```

and load one with

```
$ module load <module-name>
```

Create an *alias command* called, for example, `load-my-modules` that runs a bash script to load all the required modules in your Linux environment. Set it up so that this command is run automatically every time you log in. If you don’t need any modules yet, start by loading the modules `git` and `gcc` and come back to update your alias in a few weeks when you start needing to load more modules.

Completion: most shells support automatic completion of partially written commands. Type `load-m` and press TAB button to complete the command automatically to `load-my-modules`.

2.2 Reading and thinking activities

2.2.1 Exercise 8

Introducing Unix: <https://www.youtube.com/watch?v=tc4ROCJYbm0>. This is a 27.5 minute video that sets the scene for the work we'll be doing this semester. We will be studying xv6 (a simplified version of Unix, version 6) designed by MIT faculty. This will help you understand some of the challenges—despite being some decades old it is all still very relevant.

Also see diagram at https://en.wikipedia.org/wiki/History_of_Unix#/media/File:Unix_history-simple.svg.

2.2.2 Exercise 9

Download the xv6 book

<https://pdos.csail.mit.edu/6.S081/2022/xv6/book-riscv-rev3.pdf>

and read Chapter 1. The book is very technical and most rewarding after multiple reads. The best way is to read the chapters and mark tricky marks for yourself rather than spending time trying to understand them. After working with xv6 on some later exercises, come back to the early chapters again and check whether the material is easier to understand.

The material in this book is not examinable and each is encouraged to make a personal plan on how much time to spend on understanding the book. A lot of the material in it will help with the laboratory exercises and the summative assessment (worth 30% of the overall module mark), as well as allow to understand the practical side of the theory presented in the COMP2211 lectures.

Chapter 3

Week 2: Starting xv6

The purpose of this lab is to familiarise yourself with the xv6 operating system. The operating system is relatively small in comparison to Linux or Microsoft Windows but implements all of the basic ideas of an operating system and, more importantly, is configured in such a way that it can be easily extended and amended. Xv6 runs best on a standard Linux computer (such as feng-linux or the lab PCs), but it has been tested on an M2 Mac and the containerised version of xv6-tools will (should) run on an Intel Mac too. We haven't tested it on any form of Linux running under Windows. Please note that we can only support you running the labs on a Lab Linux PC.

3.1 Getting a copy

Start off by creating a directory for all your COMP2211 work in your Linux home directory and then change directory into it:

```
$ mkdir comp2211
$ cd comp2211
```

In that new directory, use `git` to clone a copy of the xv6 code repository:

```
$ git clone https://github.com/mit-pdos/xv6-riscv.git
  xv6_riscv_comp2211
$ cd xv6_riscv_comp2211
```

If you have a look at the structure of the xv6 directory, you will see directories shown in Table 1. Take a look at some of the files here.

In the user directory you will probably recognise some of the names of the programs. This directory contains the user space programs provided by the xv6 operating system, in particular, it contains the implementation of a simple shell, and commands like `ls`, `ln` and `echo`.

Table 3.1: xv6 root directory structure

Directory	Purpose
kernel	Implementation of the xv6 kernel
mkfs	Utility to construct a file system for xv6
user	Implementation of the xv6 user space programs

3.2 Compiling the xv6 operating system

All the tools you need to compile and build xv6 are combined (together with their dependencies) in a container (there is both a Docker and Singularity version). This means that you won't have any difficulties running the xv6 toolchain.

You can download the singularity image from the Minerva module or use a singularity command to download ("pull") a copy straight from Dockerhub. Do this in the comp2211 directory you created earlier so that the code and the tools container are in the same place

```
$ singularity pull docker://callaghanmt/xv6-tools:buildx-latest
```

This is now your toolchain containing all the utilities to get xv6 up and running. To run the container under singularity:

```
$ singularity shell xv6-tools_buildx-latest.sif
```

The container will be able to "see" the contents of your home directory, including the xv6-riscv code directory you downloaded previously.

You can now build the xv6 OS by (make sure you are in the directory containing the xv6 sources obtained earlier):

```
$ make
```

You'll see a lot of output move up the screen as the codebase compiles, then you'll get back the singularity prompt. Next, run xv6 under the qemu emulator (remember that xv6 wants to run on a RISC-V CPU which we need to emulate with qemu):

```
$ make qemu
[...]
xv6 kernel is booting
hart 1 starting
hart 2 starting
init: starting sh
$
```

At that last \$ prompt, you're now in xv6 running under qemu on an emulated RISC-V CPU. The simple shell of xv6 has started and is waiting for your commands.

You can use some of the commands you are used to on a linux machine, for example `ls`:

```
$ ls
```

.	1	1	1024
..	1	1	1024
README	2	2	2305
cat	2	3	32840
echo	2	4	31696
forktest	2	5	15832
grep	2	6	36216
init	2	7	32192
kill	2	8	31656
ln	2	9	31480
ls	2	10	34784
mkdir	2	11	31712
rm	2	12	31696
sh	2	13	54136
stressfs	2	14	32584
usertests	2	15	180480
grind	2	16	47528
wc	2	17	33792
zombie	2	18	31056
console	3	19	0

As xv6 is running in qemu it is a little challenging to quit as the input is captured by xv6 itself. To escape press ctrl-a + x (press ctrl and a at the same time, release both then press x).

The Makefile you ran to compile and emulate the riscv architecture has many build targets, we won't use all of them but the following are the ones in Table 3.2 are the ones useful here.

3.3 Xv6 shell

The xv6 shell is quite simple compared with bash, but it does support *pipes* and redirection. The syntax for the xv6 shell is similar to bash.

3.4 Exercise 1

You can write programs for the xv6 operating system in C/C++ (or indeed anything that gcc will compile to native code). In this module you will be expected to write programs in the C programming language. It might be easier sometimes to use C++ features but you should avoid this as it will be easier in the long run to use regular C.

Table 3.2: Make commands of use for this lab.

make	Purpose
make	Compiles the xv6 kernel and user space programs.
make clean	Cleans all directories of build artefacts.
make qemu	Compiles the xv6 kernel and user space programs and starts up QEMU emulating the xv6 Operating system.
make qemu-gdb	Same as above, only it also sets up remote debugging. Xv6 will pause during boot waiting for a remote debugger to attach.

Follow all the best principles of software development that you’ve learned so far. We expect that code should be well written and formatted, include lots of comments and that you use version control (such as by creating a private GitHub repository and committing new changes after each lab) to manage your work on the individual exercises.

The program you will create in this section will simply print “Hello world” to the console when it is executed. Although this seems like a simple program, due to the way xv6 works it requires a number of changes in a few different files. The program you will write will be a user program, that is it is not part of the kernel, therefore you should put your code in the user directory.

1. Create a file called `hw.c` in the user directory.
2. Populate it with the following C code.

```
#include "kernel/types.h"
#include "user/user.h"

int main(int argc, char *argv[]) {
    // Insert code here.
    // You can use print as normal.
    // Terminate the process.
    exit(0);
};
```

Looks inside the two files we include on lines 1 and 2 and think about why we need to include them.

3. Modify the Makefile to compile and include the application in the xv6 file system image. To modify the Makefile, open up the Makefile in the root directory for the git repository. Locate the line that starts with “UPROGS”; the following is a line delimited list of compiled executables that will be included in the file system image when xv6

makes its file system. Add the following to the end of the list "\$U/_hw" (make sure it is indented consistently). The Makefile will compile everything in the user directory. All executables it compiles are preceded by an underscore (you may have noticed this if you looked into the user directory once you had run make). The `hw.c` program will be compiled to an executable called "`_hw`". The line you just added to the Makefile will instruct Make to include it in the file system image.

4. Recompile xv6

```
$ make clean
$ make
```

Check that your program was included in the file system image and test your program

```
$ make qemu
...
xv6 kernel is booting
hart 1 starting
hart 2 starting
init: starting sh
$ ls
. 1 1 1024
.. 1 1 1024
README 2 2 1982
cat 2 3 22528
echo 2 4 21384
forktest 2 5 11744
grep 2 6 25912
init 2 7 22120
kill 2 8 21344
ln 2 9 21288
ls 2 10 24784
mkdir 2 11 21440
rm 2 12 21424
sh 2 13 40320
stressfs 2 14 22448
usertests 2 15 121480
wc 2 16 23688
zombie 2 17 20832
hw 2 18 21072
sleep 2 19 21096
console 3 22 0
$ hw
hello world
```

```
$
```

3.4.1 Debugging a program

Debugging your applications is probably part of your normal development workflow. Normally you would debug a program by using `gdb` or similar tool, however, because the programs you are writing are executed on an emulated machine, provided by QEMU, it becomes more difficult. Instead you will have to use a remote debugger. The GNU debugger supports remote debugging.

The rough idea is that you start the QEMU emulator, and it will pause waiting for a remote debugger to connect. In a different application you will connect your remote debugger to the QEMU emulator. At this point the QEMU emulator will continue executing the operating system and you will be able to use the remote debugger to step through your programs, set breakpoints and inspect variables. To configure xv6 for remote debugging you should use the `qemu-gdb` target in the provided make file.

```
$ make qemu-gdb
# Now run 'gdb' in another window.
$ qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel
  -m 3G -smp 3 -nographic -drive file=fs.img,if=none,format=raw,id=
    x0 -device virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0 -S -
    gdb tcp::26000
```

In a different terminal window, navigate to the root of the xv6-riscv source code and run a second copy of the xv6-tools container using singularity as above and then within that, run this command to run `gdb`:

```
gdb-multiarch -tui
```

This will open up an instance of the `gdb` debugger which is capable of cross architecture debugging. The `xv6-riscv` directory contains a file called `.gdbinit`, this file is a configuration for `gdb` and can be loaded by executing the following `gdb` command (although there won't be much to debug in this exercise).

```
(gdb) source .gdbinit
```

The configuration file tells `gdb` the architecture of the platform being debugged, the location of the object files (used for the symbol tables) and where the machine is on the network that is to be debugged. Quit `gdb` with the `q` command.

3.5 Exercise 2

Implement the UNIX program `sleep` for xv6; your `sleep` should pause for a user-specified number of ticks. A tick is a notion of time defined by the xv6 kernel, namely the time between two interrupts from the timer chip. Your solution should be in the file `user/sleep.c`. Things to consider:

- Before you start coding, read Chapter 1 of the xv6 book.
- Look at some of the other programs in `user/` (e.g., `user/echo.c`, `user/grep.c`, and `user/rm.c`) to see how you can obtain the command-line arguments passed to a program.
- If the user forgets to pass an argument, `sleep` should print an error message.
- The command-line argument is passed as a string; you can convert it to an integer using `atoi` (see `user/ulib.c`).
- Use the system call `sleep`.
- See `kernel/sysproc.c` for the xv6 kernel code that implements the `sleep` system call (look for `sys_sleep`), `user/user.h` for the C definition of `sleep` callable from a user program, and `user/usys.S` for the assembler code that jumps from user code into the kernel for `sleep`.
- Make sure `main` calls `exit()` in order to exit your program.
- Add your `sleep` program to `UPROGS` in `Makefile`; once you’ve done that, make `qemu` will compile your program and you’ll be able to run it from the xv6 shell.
- Look at Kernighan and Ritchie’s book *The C programming language* (second edition) (K&R) to learn about C.

Once completed, run the program from the xv6 shell:

```
$ make qemu
...
init: starting sh
$ sleep 10
(nothing happens for a little while)
$
```

Your solution is correct if your program pauses when run as shown above.

3.6 Exercise 3

Write a program that uses UNIX system calls to “ping-pong” a byte between two processes over a pair of pipes, one for each direction. The parent process should send a byte to the child; the child should print “<pid>: received ping”, where <pid> is its process ID, write the byte on the pipe to the parent, and exit; the parent should read the byte from the child, print “<pid>: received pong”, and exit. Your solution should be in the file `user/pingpong.c`. Things to consider:

- Use `pipe` to create a pipe.
- Use `fork` to create a child.
- Use `read` to read from the pipe, and `write` to write to the pipe.
- Use `getpid` to find the process ID of the calling process.
- User programs on xv6 have a limited set of library functions available to them. You can see the list in `user/user.h`; the source (other than for system calls) is in `user/ulib.c`, `user/printf.c`, and `user/umalloc.c`.
- When the child receives the byte, consider changing it so that, at the parent's side, we know that we are definitely printing out the received copy of the byte rather than our local copy.

Run the program from the xv6 shell and it should produce the following output:

```
$ make qemu
...
init: starting sh
$ pingpong
4:Received ping, P
3:Received pong, R
$
```

3.7 Exercise 4

Write a simple version of the UNIX `find` program: find all the files in a directory tree with a specific name. Your solution should be in the file `user/find.c`. If you haven't met the `find` command before, do a bit of reading (for example, read the output of `man find` command) and try it out on your regular Linux account first. That way you have an idea of the behaviour you are aiming for. Things to consider:

- Look at `user/ls.c` to see how to read directories.
- Use recursion to allow `find` to descend into sub-directories.
- Don't recurse into `"."` and `".."` (remember that they have a special role in Unix filesystems).
- Changes to the file system persist across runs of `qemu`; to get a clean file system run `make clean` and then `make qemu`.
- You'll need to use C strings. Have a look at K&R (the C book), for example Section 5.5, or your favourite C programming book.

- Note that `==` does not compare strings like in Python. Use `strcmp()` instead.

Your solution is correct if it produces the following output (when the file system contains the files `b` and `a/b`):

```
$ make qemu
...
init: starting sh
$ echo > b
$ mkdir a
$ echo > a/b
$ find . b
./b
./a/b
$
```

3.8 Exercise 5

Write an `uptime` program that prints the uptime in terms of ticks using the `uptime` system call.

Chapter 4

Week 3: Creating system calls and extending the xv6 shell

Before you start coding (or alongside), read Chapter 2 of the xv6 book, and Sections 4.3 and 4.4 of Chapter 4, and take a look at the following source files.

- The user-space code for system calls in `user/user.h` and `user/usys.pl`.
- The kernel-space code in `kernel/syscall.h`, `kernel/syscall.c`.
- The process-related code in `kernel/proc.h` and `kernel/proc.c`.

In the lab exercises this week we're going to explore how to do two sets of tasks:

1. Create new system calls for the xv6 OS and a corresponding user space program to access the new system call.
2. Understand more about the xv6 shell and create some more useful extensions to it.

Although there are only two programming exercises this week, they are quite involved.

4.1 Creating new system calls

In this exercise we will walk through the process of creating a new system call. Unlike the simple user programs so far, new system calls involve making changes to five different files:

- `syscall.h`
- `syscall.c`
- `sysproc.c`
- `usys.pl`

- `user.h`

Remember that a system call is a way for programs to interact with the operating system. A computer program makes a system call when it makes a request to the operating system's kernel. System calls are used for hardware services, to create or execute a process, and for communicating with kernel services, including application and process scheduling. In this very simple exercise we're going to create a new call called `GETYEAR` that will return 1975, the year that the original Unix v6 was released.

4.1.1 Step 1

Start working in the `syscall.h` file (in the `kernel` directory) where an identifying number is assigned to every system call in our xv6 system. You will see that there are 21 system calls already defined in this file. Add the following line to reserve a new system call identifying number for your own system call.

```
#define SYS_getyear 22
```

What this means is that when a program calls this `getyear` system call (with system call reference 22) a function pointed to by the function pointer `sys_getyear` will be called. We have to implement this function but as a two-step process.

1. Create a function prototype ('stub') inside `syscall.c` (you will see that all the other 21 system calls have a similar definition). Find suitable places inside this file and add the following lines:

```
extern uint64 sys_getyear(void);  
[SYS_getyear] sys_getyear,
```

2. Next, implement the system call function itself. In order to do this, open the `sysproc.c` file where system call functions are defined and add the following function.

```
// Return the year in which  
// Unix version 6 was released.  
uint64  
sys_getyear(void) {  
    return 1975;  
}
```

Notice the different coding style consistent with the style used in xv6. We are modifying an existent functions and sticking to that style. When we write new files (as above) we will use our own style—you may choose your own as well and it is worth being consistent.

4.1.2 Step 2

The final step in creating the system call is to edit the last two files, these contain the interface for your user program to access this system call. Open a Perl file `usys.pl` (in `user/`) and add this line at the end:

```
entry("getyear");
```

Next, open the file `user.h` and add the following line. This is the function that the user program will be calling.

```
int getyear(void);
```

The call to this function from a user program will be simply mapped to system call number 22 which is defined as the `SYS_getyear` preprocessor directive. The OS knows what this system call is and how to handle it. If you have completed all the previous stages, you have successfully added a new system call to xv6. However, in order to test functionality of this, you would need to add a user program which calls this system call.

4.1.3 Step 3: Creating the user program

A user program to call this new system call could be as follows:

```
#include "kernel/types.h"
#include "user/user.h"

int main(void) {
    fprintf(1, "Note: Unix V6 was released in year %d\n", getyear());
    exit(0);
}
```

To test, you'll need to add this user program (with a suitable name) to the xv6 Makefile and then rebuild xv6. You can then run under QEMU and test the program to confirm that it all works.

4.2 Exercise 1

Write a simple version of the UNIX/Linux `xargs` program: read lines from the standard input and run a command for each line, supplying the line as arguments to the command. Your solution should be in the file `user/xargs.c`. The following example illustrates `xarg`'s behavior:

```
$ echo hello too | xargs echo bye
bye hello too
$
```

Note that the command here is “`echo bye`” and the additional arguments are “`hello too`”, making the command “`echo bye hello too`”, which outputs “`bye hello too`”. Please note that `xargs` on UNIX makes an optimisation where it will feed more than one argument to the command at a time. We don’t expect you to make this optimization. To make `xargs` on UNIX behave the way we want it to for this lab, please run it with the `-n` option set to 1. For instance

```
$ echo "1\\n2" | xargs -n 1 echo line
line 1
line 2
$
```

Hints:

- Use `fork` and `exec` to invoke the command on each line of input. Use `wait` in the parent to wait for the child to complete the command.
- To read individual lines of input, read a character at a time until a newline (`'\n'`) appears.
- `kernel/param.h` declares `MAXARG`, which may be useful if you need to declare an `argv` array.
- Add the program to `UPROGS` in `Makefile`.
- Changes to the file system persist across runs of `qemu`; to get a clean file system run `make clean` and then `make qemu`.

How do we know it works? You could test it alongside the `find` command you created last week:

```
$ mkdir a
$ echo hello > a/b
$ mkdir c
$ echo hello > c/b
$ echo hello > b
$ find . b
./a/b
./c/b
./b
$ find . b | xargs grep hello
hello
hello
hello
```

If you didn’t complete the `find` exercise from last week, then all is not lost. You could actually implement this outside of `xv6` in the normal shell and use the regular Linux `find` command. If you find the C programming a bit too difficult at this stage you could even implement it in Python to get the steps right and then reimplement in C afterwards.

4.3 Exercise 2

In this task you are going to extend the xv6 shell (implemented in `sh.c`) by adding functionality to provide command-line `history`. If you have not met `history` in modern Linux then first do a bit of investigation and then explore how it works on your regular lab machine. You'll need to provide both the functionality to store commands as they are entered into a file and then a separate user program called `history` that will dump the contents of that file to the terminal. The history command will line-number the history of commands and it will normally be able to take a single integer parameter to restrict the number of commands that are printed out, so

```
$ history 10
```

for example, would print out the last 10 commands. Normally, `history` wouldn't store an unlimited number of previous commands, perhaps just the last 500. You may want to implement this feature via a system call (or calls) but it can be accomplished completely in a user program and via additions to `sh.c`.

Reminder: the command `man` in Linux can be used to easily find information about other commands. For example, run `man history` and then use the space button to go through the documentation page by page or press `/` to search. Press `q` to exit. Also try `man man`.

Chapter 5

Week 4: Creating a small shell interface

You must submit your work to the appropriate submission point in Gradescope, which will be automatically marked. You should submit a single file called `my_shell.c`. Any other files you submit will not be marked. Although you do not need to include any additional supporting documentation or report, we do expect that your code is well written, tested and commented.

Deadline: Week 6 of teaching. Tuesday. 7th of November, 2023. 14:00.
Weighting: 30% of the final module mark.

In this coursework you will demonstrate:

- An understanding of how processes are created by the operating system.
- An understanding of file descriptors and their relationship to pipes and redirection.
- The ability to program components of an operating system.

Exercise

In this coursework you will implement a simple shell for the xv6 operating system. This new shell will be implemented as a user space program. Before you attempt this coursework, make sure you have gone through most of the formative assessment exercise in the preceding weeks and convinced yourself that you know how various parts work. You should provide your implementation in a new file called `my_shell.c`. You may use any helper functions provided by the xv6 kernel or user libraries. For each of the following items implement the feature into your shell, as you progress the features to implement become harder. Your implementation should not require you to modify any file other than `my_shell.c` and the `Makefile`.

Part 1

To start with create a file in the user directory for the Xv6 operating system source code called `my_shell.c`.

1. Insert the appropriate headers and a main function into your `my_shell.c`.
2. Modify the Makefile to include your shell in the build process.
3. Test that your new executable is being included by running `make qemu`. You should be able to execute your new program by typing `myshell` at the command prompt.

Part 2: Execute simple commands (5 Marks)

Implement the execution of simple commands. Your shell should be able to:

- Prompt the user for a command by printing “>>>” as a command prompt.
- Execute a command inputted to the command prompt.
- Loop indefinitely until the shell is exited.
- Handle the “`cd`” command—you will notice that this command will need to be treated as a special case.

Do not forget to stress-test your simple shell before moving on to advanced features. The automatic marking will be testing it on various cases and marks will be deducted if it does not work when the same command is provided in a different format, such as with extra spacing. For example, consider (note the amount and location of space characters which may impact the shell if they are not detected):

```
$ echo hello world
$   echo hello      world
```

Once you are comfortable that you have tested your shell with any possible command that could reveal bugs, move on to implement the following advanced features.

Part 3: Input/Output redirection (6 Marks)

Implement Input/Output redirection. Your shell should be able to handle two element redirections. For example,

```
$ echo "Hello world" > temp
$ cat < temp
```

Part 4: Pipes (6 Marks)

Implement pipes. Your shell should be able to handle two element pipelines. For example,

```
$ cat README | grep github
```

Part 5: Additional features (8 Marks)

Implement the following advanced features:

1. Implement multi-element pipelines. For example,

```
$ ls | grep test | cat
```

2. Implement non-trivial combinations of pipes and redirection. For example,

```
$ ls | grep test | cat > myoutput
```

3. Implement the “;” operator that allows a list of shell commands to be given and executed sequentially.

```
$ ls | grep test | cat > myoutput; cat myoutput
```

Submission

Submission will be through gradescope. We will announce the instructions on Minerva.

Chapter 6

Week 7: Exploring memory in regular Linux

In the lab exercises this week, we are going to temporarily leave xv6 on one side and explore the Linux operating system on a lab PC (or `feng-gps` , `feng-linux` or any other Linux computer you have access to). If you have any remaining time this week, please use it to complete the previous week's labs to the best of your abilities. Objectives this week are:

- To get an introduction and start to explore virtual memory.
- Understand how to view memory related aspects of running processes in Linux. It might be worth recording your findings in this lab for future reference (in lectures and for exam revision).

6.1 Linux memory management

1. Use the Linux tools `free` and `vmstat -a` to find memory usage values (including units) and note them down using the table below. What do the terms *physical memory* and *virtual memory* refer to?

Physical memory	Virtual memory
Total	Free
Used	Inactive
Available	Active
Buffers	

2. Open a terminal window and start `top`. Look at the output of `top` and record the value of RES for the `top` process (including units). What does the RES value represent?
3. Briefly describe how to specify sorting on virtual memory used with the `top` command.

4. What program running at the moment needs the most virtual memory?
5. Explore the `/proc/vmstat` pseudofile and find the section for *pagefaults*. What value does it have?
6. What is a *pagefault*?
7. What is the difference between a *pgfault* and a *pgmajfault*?
8. Take a look at the file `/proc/meminfo`. What do the following parameters represent: *Dirty*, *Mapped*, *PageTables*, *Hugepagesize*?
9. Run the command `pmap -x <pid>` (replace `<pid>` with the pid of a running process on your system, Firefox say). What does this command show? Explain the format of the output with the `-x` option.

6.2 Exploring memory usage

When talking about memory, we are referring either to the CPU's cache or main memory (i.e., RAM). We're interested in the latter in the tasks that follow. Put this Python code into a file, and save as `ex01.py`.

```
import time
def do_append(size):
    result = []
    for i in range(size):
        message= "some unique object %d" % ( i, ) * 1000
        result.append(message)
        time.sleep(0.0001)
    return result
def do_allocate(size):
    result=size*[None]
    for i in range(size):
        message= "some unique object %d" % ( i, ) * 1000
        result[i]= message
        time.sleep(0.0001)
    return result
if __name__ == '__main__':
    size = 100000
    #
    # Experiment 1
    #
    #do_append(size)
    #
```

```
# Experiment 2
#
#do_allocate(size)
#
# Experiment 3
#
#do_append(size)
#do_allocate(size)
#
# Experiment 4
#
#result1 = do_append(size)
#result2 = do_allocate(size)
```

Take a look at the code. What is the difference between the methods `do_append()` and `do_allocate()`?

Use the `vmstat` command (possibly in conjunction with the `w` parameter) to monitor the system memory usage while performing the following experiments. In the main method, call:

- The `do_append` method on its own (see Experiment 1).
- The `do_allocate` method on its own (see Experiment 2).
- Both methods as shown in the Experiment 3 area in the code.
- Both methods as shown in the Experiment 4 area in the code.

What do you think is going on here? Change the code in `do_append()` so that the memory usage will be the same when performing Experiment 3 or Experiment 4.

Chapter 7

Week 8: Exploring linked lists in C

In the lab exercises this week, we are leaving xv6 on one side again and explore probably the most important C data structure for memory management—the linked list. You may have met this already in your first year at the School; in this Lab you will implement a linked list and ensure you can add and remove nodes and display the data stored in each node in the list.

Firstly, if you need a reminder of what a linked list is then read about it in your favourite algorithms book or check out tutorials online.

7.1 Exercise

Firstly, get hold of the Github repo that contains a demonstration executable and a starter template:

```
$ git clone https://github.com/COMP2211/linked-lists.git
```

Explore the demonstration and alongside the tutorials, make sure you are happy with the basic concepts of the linked list. Secondly, you are going to have a go at writing some functions to perform some of these tasks yourself. Remember that a linked list is a dynamic data structure so you'll need to remember to use `malloc` along the way. So, using a similar struct to represent each node as the tutorials use (you don't have to use this, but it's a good starting point and does not need to be any more complicated than a data node and a pointer to the next node in the list):

```
// A linked list node
struct Node {
    int data;
    struct Node* next;
};
```

Open up the file `linked_list.c`—you will see it has a basic menu structure, a place for you to define your own `struct` and some stub functions. You will need to complete the functions to:

1. Initialise a four element linked list with the data field set to 1,2,3,4 for those first four fields respectively (either hard coded or request the user to type in those nodes).
2. Create a function to display the elements of the list in turn.
3. Create a function to add an element to the list at the end.
4. Create a function to add an element to the list at the beginning.
5. Create a function to remove an element from the list at a position that the user specifies.

Take your time on this and attempt as many as you can. It will help with the final exercise.

Chapter 8

Weeks 9–11: Debugging dynamic memory management on xv6

This week we will be looking at an implementation of dynamic memory management on the xv6 operating system: functions `malloc` and `free`, which you might have used in C before. An incomplete implementation is provided and the task is to integrate it, test its functionalities, find out the problems in the code, fix them, and also extend it with various optimisations.

Clone the repository containing the code for this part into your COMP2211 working directory.

```
$ git clone https://github.com/mmikaitis/COMP2211-2023-24.git
```

copy `malloc_stress.c`, `memory_management.c` and `memory_management.h` into `user/` directory of your copy of xv6. Make the `malloc_stress` a user program runnable from the command line, as in previous tasks.

8.1 Exercise 1: Debug malloc and free

The supplied memory management implementation has several bugs. Run `malloc_stress` in order to discover where `user/memory_management.c` fails the requirements. A basic implementation should pass all the tests, except `test_malloc_align`, `test_alloc_give_back`, `test_sbrk_opt` as these test the advanced features implemented in the next section.

Once you are finished debugging and still can't pass the tests, talk to the lab demonstrators for ideas.

The fastest way to find bugs in the code is by understanding what it is meant to do and what it actually does. You may find `sbrk(0)` useful, which returns the current program break pointer. Also, the supplied `memory_management.c` has a useful function `print_mem_blocks()`.

8.2 Exercise 2: Implement optimisations

Implement the following three optimisations to the dynamic memory management of xv6.

1. 16-byte memory alignment.
2. Consider how many bytes of physical memory is allocated at minimum with a call to `sbrk`. Check `kernel/kalloc.c` which `sbrk` uses. This should give ideas for various optimisations.
3. If a user requests to free the last block in the list, return the memory to OS with `sbrk(-size)`.

Check if your implementation passes the remaining stress tests in `malloc_stress.c`. Tell the lab demonstrators your ideas and see if you can come up with further improvements based on information you get from them. Once done, check that all of the 17 tests in `malloc_stress.c` pass.

Chapter 9

Conclusion

This concludes the COMP2211 laboratory exercises. Congratulations on distinguishing yourself by working through these, improving your Bash and C programming skills, and looking into the internals of the xv6 OS. Thank you for spending your time.